

```
=====
BIBLE PYTHON-GSMMODEM & GSMTERMLIB – RÉFÉRENCE EXHAUSTIVE
LÉGENDE : [gm] = GsmModem | [sms] = SMS | [call] = appels | [ussd] = USSD
          [at] = AT commands | [ex] = exceptions | [cli] = outils CLI
          [gt] = gsmtermlib | [obj] = objets retournés
=====
```

△ PACKAGES :

- python-gsmmodem : dépôt original (faucamp) – Python 2/3
- python-gsmmodem-new: fork maintenu (babca) – Python 3 recommandé □

Les deux ont la même API – seul le nom du package PyPI diffère.

[util] Installation et imports

DESCRIPTION : Module Python pour contrôler un modem GSM connecté en série
POURQUOI : Envoyer/recevoir des SMS, gérer des appels, lancer des sessions USSD.
CONTEXTE : IoT, Raspberry Pi, automatisation, alertes SMS, serveurs de SMS.

INSTALLATION :

pip install python-gsmmodem-new	Fork maintenu (recommandé)
pip install python-gsmmodem	Version originale

PRÉREQUIS MATÉRIELS :

- # Modem GSM connecté via USB ou UART (SIM800, SIM900, Huawei, Quectel, ZTE...)
- # Carte SIM insérée dans le modem
- # Port série disponible : /dev/ttyUSB0, /dev/ttyS0, COM3...
- # Permissions : ajouter l'utilisateur au groupe dialout
- # sudo usermod -a -G dialout \$USER

DÉCOUVERTE DU PORT :

- # Avant branchement : ls /dev/tty*
- # Après branchement : ls /dev/tty* → noter le nouveau /dev/ttyUSBx
- # Identifier le modem : python -m gsmmodem.tools.identify_modem

IMPORTS PRINCIPAUX :

from gsmmodem.modem import GsmModem	Classe principale
from gsmmodem.modem import SentSms, ReceivedSms	Objets SMS
from gsmmodem.modem import StatusReport	Rapport de livraison
from gsmmodem.modem import IncomingCall, Call	Objets appel
from gsmmodem.modem import Ussd	Objet USSD
from gsmmodem.exceptions import (	
CommandError, CmeError, CmsError,	
TimeoutException, InvalidStateException,	
PinRequiredError, IncorrectPinError,	
InterruptedException, SmscNumberUnknownError,	
EncodingError	
)	

[gm] GsmModem – connexion et initialisation

DESCRIPTION : Classe principale pour contrôler un modem GSM via le port série
POURQUOI : Point d'entrée de toutes les opérations (SMS, appels, USSD, AT).
CONTEXTE : Se connecte via pySerial et gère les notifications asynchrones.

CONSTRUCTEUR :

GsmModem(port, baudrate=115200,	
incomingCallCallbackFunc=None,	
smsReceivedCallbackFunc=None,	
smsStatusReportCallback=None)	
GsmModem("/dev/ttyUSB0")	Port série, baudrate par défaut
GsmModem("/dev/ttyUSB0", 9600)	Port + baudrate spécifique
GsmModem("COM3", 115200)	Windows
GsmModem("/dev/ttyUSB0",	
incomingCallCallbackFunc=handleCall,	Callback appel entrant
smsReceivedCallbackFunc=handleSms,	Callback SMS reçu
smsStatusReportCallback=handleReport)	Callback rapport de livraison

— CONNEXION / DÉCONNEXION —

modem.connect()	Connexion sans PIN SIM
modem.connect(pin="1234")	Connexion avec déverrouillage PIN
modem.connect(waitingForModemToStartInSeconds=0)	Sans attente de démarrage

<code>modem.close()</code>	Ferme la connexion (toujours appeler)
— ATTENTE DE COUVERTURE RÉSEAU —	
<code>modem.waitForNetworkCoverage()</code>	Bloque jusqu'à avoir du réseau
<code>modem.waitForNetworkCoverage(timeout=30)</code>	Attente max en secondes
— INFORMATIONS DU MODEM —	
<code>modem.manufacturer</code>	Fabricant (str) – ex: "HUAWEI"
<code>modem.model</code>	Modèle (str) – ex: "E3131"
<code>modem.revision</code>	Révision firmware (str)
<code>modem.imei</code>	IMEI du modem (str, 15 chiffres)
<code>modem.imsi</code>	IMSI de la SIM (str, 15 chiffres)
<code>modem.networkName</code>	Nom de l'opérateur réseau (str)
<code>modem.signalStrength</code>	Niveau de signal (int, 0-31, 99=inconnu)
<code>modem.ownNumber</code>	Numéro de la SIM (str, si disponible)
<code>modem.smsc</code>	Numéro du SMSC (str)
<code>modem.smsTextMode</code>	Vrai si mode texte (False = PDU)
<code>modem.alive</code>	Vrai si la connexion est active
— PING —	
<code>modem.write("AT")</code>	Ping simple – retourne ["OK"]

EXEMPLES :

```
from gsmmodem.modem import GsmModem
from gsmmodem.exceptions import PinRequiredError, IncorrectPinError

modem = GsmModem("/dev/ttyUSB0", 115200)
try:
    modem.connect(pin="1234")
except PinRequiredError:
    print("PIN requis")
except IncorrectPinError:
    print("PIN incorrect")

print(f"Opérateur : {modem.networkName}")
print(f"Signal      : {modem.signalStrength}/31")
print(f"IMEI         : {modem.imei}")

modem.close()
```

[sms] Envoi de SMS – `modem.sendSms`

DESCRIPTION : Envoie un SMS à un numéro de destination
 POURQUOI : Envoyer des alertes, notifications, OTP ou messages quelconques.
 CONTEXTE : Automatisation, alertes IoT, 2FA, communication machine-à-machine.

FONCTIONS PRINCIPALES :

<code>modem.sendSms(destination, text)</code>	Envoi simple
<code>modem.sendSms("+33612345678", "Bonjour !")</code>	Format international recommandé
<code>modem.sendSms(destination, text, waitForDeliveryReport=False, deliveryTimeout=15, sendFlash=False)</code>	Ne pas attendre l'accusé de réception Timeout accusé de réception (sec) Flash SMS (affiché sans sauvegarde)

— AVEC ACCUSÉ DE RÉCEPTION —

```
sms = modem.sendSms(dest, text, waitForDeliveryReport=True)
sms = modem.sendSms(dest, text, waitForDeliveryReport=True, deliveryTimeout=30)
```

— FLASH SMS —

```
modem.sendSms(dest, text, sendFlash=True) Affiché immédiatement, non sauvegardé
```

— OBJET SentSms RETOURNÉ —

<code>sms = modem.sendSms(dest, text)</code>	
<code>sms.number</code>	Numéro destinataire
<code>sms.text</code>	Texte du message
<code>sms.reference</code>	Référence interne du SMS (int)
<code>sms.status</code>	Statut : ENROUTE, DELIVERED, FAILED
<code>sms.report</code>	Objet StatusReport (si livraison reçue)
<code>SentSms.ENROUTE</code>	Constante : en transit
<code>SentSms.DELIVERED</code>	Constante : livré
<code>SentSms.FAILED</code>	Constante : échoué

EXEMPLES :

```

from gsmmodem.modem import GsmModem, SentSms
from gsmmodem.exceptions import TimeoutException

modem = GsmModem("/dev/ttyUSB0")
modem.connect()
modem.waitForNetworkCoverage(30)

# Envoi simple
sms = modem.sendSms("+33612345678", "Alerte : capteur actif !")
print(f"SMS envoyé, ref={sms.reference}, statut={sms.status}")

# Avec accusé de réception
try:
    sms = modem.sendSms("+33612345678", "Test livraison",
                        waitForDeliveryReport=True, deliveryTimeout=30)
    if sms.status == SentSms.DELIVERED:
        print("☐ SMS livré")
    else:
        print("☐ Échec de livraison")
except TimeoutException:
    print("Timeout – pas d'accusé reçu")

modem.close()

```

[sms] Réception de SMS – smsReceivedCallbackFunc

DESCRIPTION : Reçoit les SMS entrants via un callback asynchrone
 POURQUOI : Réagir en temps réel aux SMS sans polling actif.
 CONTEXTE : Gateway SMS, commandes à distance, réponses automatiques, IoT.

PATTERN CALLBACK :
 def handleSms(sms): Appelé dans un thread séparé
 ...

```

modem = GsmModem("/dev/ttyUSB0",
    smsReceivedCallbackFunc=handleSms)

```

— OBJET ReceivedSms —

sms.number	Numéro de l'expéditeur (str)
sms.text	Texte du message (str)
sms.time	Date/heure d'envoi (datetime)
sms.smsc	Numéro SMSC (str, si disponible)
sms.isPartial	Vrai si SMS multi-parties incomplet
sms.udh	User Data Header (PDU mode)
sms.status	Statut (REC UNREAD, REC READ...)
sms.storage	Mémoire de stockage ("SM" ou "ME")
sms.index	Index en mémoire SIM/modem

— LECTURE DES SMS STOCKÉS —

modem.listStoredSms(status=None)	Liste tous les SMS stockés
modem.listStoredSms(status="REC UNREAD")	Seulement les non lus
modem.listStoredSms(status="REC READ")	Seulement les lus
modem.listStoredSms(status="ALL")	Tous
modem.getStoredSms(index)	Récupère un SMS par son index
modem.deleteStoredSms(index)	Supprime un SMS par son index
modem.deleteMultipleStoredSms(statusCode=0)	Supprime par statut (0=lus)

EXEMPLES :

```

import time
from gsmmodem.modem import GsmModem

def handleSms(sms):
    print(f"SMS reçu de {sms.number} : {sms.text}")
    if sms.text.strip().lower() == "ping":
        modem.sendSms(sms.number, "PONG")

modem = GsmModem("/dev/ttyUSB0", smsReceivedCallbackFunc=handleSms)
modem.connect()
modem.waitForNetworkCoverage()
print("En attente de SMS... (Ctrl+C pour arrêter)")
try:
    while True:
        time.sleep(1)

```

```
except KeyboardInterrupt:
    modem.close()

# Lecture des SMS stockés
sms_list = modem.listStoredSms(status="REC UNREAD")
for sms in sms_list:
    print(f"{sms.number}: {sms.text}")
    modem.deleteStoredSms(sms.index)
```

[obj] Rapport de livraison – StatusReport / smsStatusReportCallback

DESCRIPTION : Accusé de réception de livraison d'un SMS
POURQUOI : Confirmer que le SMS a bien été reçu par le destinataire.
CONTEXTE : Applications critiques, traçabilité des messages.

PATTERN CALLBACK :

```
def handleReport(report):
    ...
modem = GsmModem("/dev/ttyUSB0", smsStatusReportCallback=handleReport)
```

OBJET StatusReport

report.reference	Référence du SMS (relie à SentSms)
report.number	Numéro destinataire
report.timeSent	datetime d'envoi
report.timeFinalized	datetime de livraison/échec
report.deliveryStatus	Statut : DELIVERED ou FAILED
report.smsc	Numéro SMSC
StatusReport.DELIVERED	Constante : livré avec succès
StatusReport.FAILED	Constante : livraison échouée

EXEMPLES :

```
def handleReport(report):
    status = "✅ Livré" if report.deliveryStatus == StatusReport.DELIVERED else "❌ Échoué"
    print(f"SMS ref={report.reference} → {status} à {report.timeFinalized}")
```

[call] Appels vocaux – makeCall / incomingCallCallbackFunc

DESCRIPTION : Passe des appels sortants et gère les appels entrants
POURQUOI : Automatisation d'appels vocaux, IVR simple, alertes sonores.
CONTEXTE : Systèmes d'alarme, notifications d'urgence, tests de couverture.

APPELS ENTRANTS

```
def handleCall(call):
    if call.ringCount == 1:
        call.answer()
    elif call.ringCount == 3:
        call.hangup()

modem = GsmModem("/dev/ttyUSB0",
    incomingCallCallbackFunc=handleCall)
```

	Callback appelé dans un thread séparé
	Premier appel
	Décroche
	Raccroche

OBJET IncomingCall

call.number	Numéro de l'appelant (str)
call.callerName	Nom de l'appelant (str, si dispo)
call.type	Type d'appel (int: 0=voix, 1=données)
call.ringCount	Nombre de sonneries
call.answered	Vrai si l'appel est décroché
call.answer()	Répond à l'appel
call.hangup()	Raccroche
call.sendDtmf(tones)	Envoie des tons DTMF ("123#")

APPELS SORTANTS

```
call = modem.dial(number)
call = modem.dial("+33612345678")
call = modem.dial(number, waitForAnswer=True)
call.waitForCallInitUpdate(timeout=30)
call.hangup()
call.isActive()
call.sendDtmf("1234#")
```

	Passe un appel vocal
	Numéro au format international
	Attend la réponse (bloquant)
	Attend l'état initial de l'appel
	Raccroche l'appel sortant
	Vrai si l'appel est actif
	Envoie des tonalités DTMF

OBJET Call (sortant)

call.number	Numéro composé
call.answered	Vrai si l'appel a été accepté
call.isActive()	Vrai si actif
call.hangup()	Raccroche

EXEMPLES :

```
# Appel entrant avec réponse automatique
import time
from gsmmodem.modem import GsmModem

def handleCall(call):
    print(f"Appel de {call.number}, sonnerie #{call.ringCount}")
    if call.ringCount == 2:
        call.answer()
        time.sleep(3)                # Garder en ligne 3 sec
        call.hangup()

modem = GsmModem("/dev/ttyUSB0", incomingCallCallbackFunc=handleCall)
modem.connect()
try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    modem.close()

# Appel sortant
call = modem.dial("+33612345678", waitForAnswer=True)
if call.answered:
    print("Appel en cours")
    time.sleep(5)
call.hangup()
```

[ussd] Sessions USSD – modem.sendUssd

DESCRIPTION : Lance une session USSD (Unstructured Supplementary Service Data)
 POURQUOI : Consulter le solde, activer des options, interroger l'opérateur.
 CONTEXTE : Crédit prépayé (*100#), menus opérateur, transferts d'argent mobile.

FONCTIONS PRINCIPALES :

response = modem.sendUssd(ussdString)	Lance une session USSD
response = modem.sendUssd("*100#")	Vérification de solde (exemple)
response = modem.sendUssd(ussdString, responseTimeout=15)	Timeout en secondes

— OBJET Ussd RETOURNÉ —

response.message	Réponse textuelle de l'opérateur (str)
response.sessionActive	Vrai si la session est encore ouverte
response.reply(text)	Répond dans la session active
response.cancel()	Ferme la session USSD

EXEMPLES :

```
from gsmmodem.modem import GsmModem
from gsmmodem.exceptions import TimeoutException

modem = GsmModem("/dev/ttyUSB0")
modem.connect()
modem.waitForNetworkCoverage(30)

# Consulter le solde
try:
    response = modem.sendUssd("*100#")
    print(f"Réponse USSD : {response.message}")
    if response.sessionActive:
        response.cancel()
except TimeoutException:
    print("Timeout USSD")

# Session interactive (menu)
response = modem.sendUssd("*123#")
print(response.message)                # Menu principal
if response.sessionActive:
    response2 = response.reply("1")    # Choisir option 1
    print(response2.message)
    if response2.sessionActive:
```

```
response2.cancel()
```

```
modem.close()
```

```
[at] Commandes AT bas niveau – modem.write
```

DESCRIPTION : Envoie des commandes AT directement au modem
POURQUOI : Accéder à des fonctionnalités non couvertes par l'API haut niveau.
CONTEXTE : Débogage, commandes propriétaires, configuration avancée.

FONCTIONS PRINCIPALES :

<code>modem.write(command)</code>	Envoie une commande AT
<code>modem.write("AT")</code>	Commande ping → ["OK"]
<code>modem.write("AT+CSQ")</code>	Signal → ["+CSQ: 20,99", "OK"]
<code>modem.write("AT+CGMI")</code>	Fabricant → ["HUAWEI", "OK"]
<code>modem.write("AT+CGMM")</code>	Modèle → ["E3131", "OK"]
<code>modem.write("AT+CGSN")</code>	IMEI → ["358...", "OK"]
<code>modem.write("AT+CIMI")</code>	IMSI → ["208...", "OK"]
<code>modem.write("AT+CREG?")</code>	État réseau → ["+CREG: 0,1", "OK"]
<code>modem.write("AT+COPS?")</code>	Opérateur actuel
<code>modem.write("AT+CPAS")</code>	État du modem (0=prêt, 3=sonne)
<code>modem.write("AT+CFUN?")</code>	Fonctionnalité (1=plein, 0=min)
<code>modem.write("AT+CMEE=1")</code>	Active les erreurs détaillées
<code>modem.write("AT+CLVL?")</code>	Niveau haut-parleur
<code>modem.write("ATZ")</code>	Reset soft du modem

— PARAMÈTRES DE WRITE —

<code>modem.write(command,</code>	
<code> waitForResponse=True,</code>	Attend la réponse (défaut)
<code> timeout=None,</code>	Timeout en secondes
<code> parseError=True,</code>	Lève CommandError si erreur
<code> writeTerm="\r",</code>	Terminateur d'écriture
<code> expectedResponseTermSeq=None)</code>	Terminateur de réponse attendu

<code>modem.write("AT+CMGS=...", timeout=3,</code>	
<code> expectedResponseTermSeq="> ")</code>	Attendre le prompt SMS

— COMMANDES AT STANDARDS UTILES —

AT	Test de présence → OK
ATZ	Reset du modem
ATE0	Désactive l'écho
ATE1	Active l'écho
AT+CFUN=0	Fonctionnalité minimale
AT+CFUN=1	Fonctionnalité maximale (reset)
AT+CPIN?	État du PIN SIM
AT+CSQ	Niveau de signal (0-31)
AT+CREG?	État d'enregistrement réseau
AT+COPS?	Opérateur sélectionné
AT+COPS=0	Sélection automatique opérateur
AT+CMGF=0	Mode PDU
AT+CMGF=1	Mode texte
AT+CPMS?	Mémoire SMS disponible
AT+CMGD=1,4	Supprime tous les SMS
AT+CLAC	Liste des commandes supportées

EXEMPLES :

```
# Lire le signal
reponse = modem.write("AT+CSQ")
# reponse = ["+CSQ: 20,99", "OK"]
signal = int(reponse[0].split(": ")[1].split(",")[0])
print(f"Signal : {signal}/31")

# Lire l'état du réseau
reponse = modem.write("AT+CREG?")
# "+CREG: 0,1" → 1 = enregistré réseau local, 5 = itinérance

# Lister les commandes disponibles
cmds = modem.write("AT+CLAC")
print(cmds)
```

```
[ex] Exceptions – gsmmodem.exceptions
```

DESCRIPTION : Exceptions levées par python-gsmmodem
POURQUOI : Distinguer les erreurs matérielles, réseau, PIN et timeout.
CONTEXTE : Gestion robuste des erreurs dans les applications de production.

HIÉRARCHIE :

Exception	
├── gsmmodem.exceptions.CommandError	Erreur de commande AT générique
│ ├── CmeError	Erreur CME (Mobile Equipment)
│ └── CmsError	Erreur CMS (SMS)
├── TimeoutException	Timeout de réponse du modem
├── InvalidStateException	État invalide pour l'opération
├── InterruptedException	Opération interrompue
├── PinRequiredError	PIN SIM requis
├── IncorrectPinError	PIN SIM incorrect
├── SmscNumberUnknownError	Numéro SMSC inconnu
└── EncodingError	Erreur d'encodage PDU/texte

DÉTAILS

CommandError(command, errorCode)	Erreur AT générique
e.command	Commande AT ayant échoué
e.errorCode	Code d'erreur numérique
CmeError(errorCode)	Erreur CME (+CME ERROR: N)
# Codes CME fréquents :	
# 3 = Opération non autorisée	
# 4 = Opération non supportée	
# 10 = SIM non insérée	
# 11 = PIN SIM requis	
# 12 = PUK SIM requis	
# 13 = SIM en échec	
# 16 = Mot de passe incorrect	
# 30 = Réseau absent	
CmsError(errorCode)	Erreur CMS (+CMS ERROR: N)
# Codes CMS fréquents :	
# 1 = Mémoire SMS pleine	
# 321 = Index SMS invalide	
# 322 = Mémoire pleine (SIM)	
# 330 = Centre SMS inconnu	
# 500 = Erreur inconnue	
TimeoutException(data)	Réponse non reçue dans le délai
e.args[0]	Données partielles reçues avant timeout
PinRequiredError	SIM verrouillée par PIN
IncorrectPinError	PIN erroné (risque de blocage PUK)

EXEMPLES :

```
from gsmmodem.exceptions import (
    CommandError, CmeError, CmsError, TimeoutException,
    PinRequiredError, IncorrectPinError
)

try:
    modem.connect(pin="1234")
except PinRequiredError:
    print("PIN requis")
except IncorrectPinError:
    print("⚠ PIN incorrect ! Risque de blocage PUK")

try:
    modem.sendSms("+33612345678", "Test")
except CmsError as e:
    if e.errorCode == 322:
        print("Mémoire SMS pleine")
    else:
        print(f"Erreur CMS {e.errorCode}")
except CmeError as e:
    print(f"Erreur CME {e.errorCode}")
except TimeoutException:
    print("Timeout modem")
except CommandError as e:
    print(f"Erreur commande : {e}")
```

[util] Logging et débogage

DESCRIPTION : Activation des logs pour diagnostiquer les problèmes
POURQUOI : Voir les échanges AT bruts pour déboguer.
CONTEXTE : Mise au point, modems non supportés, comportements inattendus.

FONCTIONS PRINCIPALES :

```
import logging
logging.basicConfig(format="%(levelname)s: %(message)s", level=logging.DEBUG)

# Logs spécifiques à gsmmodem uniquement
logging.getLogger("gsmmodem").setLevel(logging.DEBUG)
logging.getLogger("gsmmodem.serial_comms").setLevel(logging.DEBUG) # Bytes bruts
```

— IDENTIFY-MODEM —

```
# Outil de diagnostic fourni avec le package
python -m gsmmodem.tools.identify_modem          Identifie et affiche les infos du modem
python -m gsmmodem.tools.identify_modem -p /dev/ttyUSB0  Port explicite
```

[cli] Outils CLI – sendsms / identify-modem

DESCRIPTION : Utilitaires en ligne de commande fournis avec python-gsmmodem
POURQUOI : Tester rapidement le modem sans écrire de code Python.
CONTEXTE : Tests de déploiement, scripts shell, diagnostics terrain.

— SENDSMS —

```
python tools/sendsms.py --port /dev/ttyUSB0 --destination +33612345678 --msg "Texte"
python tools/sendsms.py -p /dev/ttyUSB0 -d +33612345678 -m "Texte"
python tools/sendsms.py --port /dev/ttyUSB0 --pin 1234 -d +33... -m "Texte"
python tools/sendsms.py --port /dev/ttyUSB0 --baudrate 9600 -d +33... -m "Texte"
```

— IDENTIFY-MODEM —

```
python tools/identify_modem.py          Détecte et identifie le modem
python tools/identify_modem.py -p /dev/ttyUSB0  Port explicite
# Affiche : fabricant, modèle, révision, IMEI, état SIM, signal
```

— GSMTERM —

```
python tools/gsmterm.py          Terminal AT interactif
python tools/gsmterm.py -p /dev/ttyUSB0 -b 115200
# Fonctionnalités :
# - Complétion automatique des commandes AT (Tab)
# - Aide intégrée pour les commandes AT courantes
# - Historique des commandes (flèches)
# - Prompt contextuel (port, baudrate)
```

GSMTERMLIB

[gt] gsmttermlib – bibliothèque du terminal GSM interactif

DESCRIPTION : Sous-module de python-gsmmodem fournissant le terminal AT interactif
POURQUOI : Facilite l'exploration et le débogage des modems GSM via AT.
CONTEXTE : Outil de développement inclus dans tools/gsmttermlib/.

LOCALISATION :

```
tools/gsmttermlib/
├── atcommands.py    Base de données des commandes AT avec aide intégrée
├── tty.py           Abstraction du port série TTY
└── __init__.py
```

— ATCOMMANDS.PY – BASE DE DONNÉES AT —

```
# Dictionnaire de commandes AT avec catégories, paramètres et descriptions
# Catégories :
# c[0] = Informations générales / identification
# c[1] = Appels vocaux
# c[2] = Réseau et enregistrement
# c[3] = Phonebook
# c[4] = SIM / PIN
```

```
# c[5] = SMS
# c[6] = GPRS / data

# Commandes répertoriées avec aide :
AT+CGMI      Fabricant – Product Manufacturer Identification
AT+CGMM      Modèle – Product Model Number
AT+CGMR      Révision firmware
AT+CGSN      IMEI – Product Serial Number
AT+CSCS      Jeu de caractères TE
AT+WPCS      Jeu de caractères phonebook
AT+CPOF      Power Off
AT+CFUN      Fonctionnalité modem (0=min, 1=max)
AT+CEER      Rapport d'erreur étendu
AT+CLAC      Liste des commandes AT disponibles
ATA          Répondre à un appel entrant
ATH          Raccrocher
ATH0         Raccrocher
ATH1         Raccrocher appel sortant (pendant numérotation)
ATD          Composer un numéro
AT+CLIR      Restriction d'ID appelant
AT+VTD       Durée des tonalités DTMF
AT+VTS       Envoyer des tonalités DTMF
AT+CRSM      Generic SIM Access
AT+CIMI      IMSI – International Mobile Subscriber Identity
AT+CNUM      Numéro de l'abonné
AT+CPIN      État du PIN SIM
AT+CPWD      Changer le PIN
AT+CLCK      Verrouillage de facilité
AT+CREG      État d'enregistrement réseau
AT+COPS      Sélection opérateur
AT+CSQ       Qualité du signal (0-31)
AT+CMEE      Rapport d'erreurs étendu (+CME)
AT+CPMS      Mémoire préférée SMS
AT+CMGF      Format SMS (0=PDU, 1=texte)
AT+CSMS      Type de service SMS
AT+CSCA      Centre de service SMS
AT+CMGS      Envoyer un SMS
AT+CMGL      Lister les SMS stockés
AT+CMGR      Lire un SMS par index
AT+CMGD      Supprimer un SMS
AT+CMGW      Écrire un SMS en mémoire
AT+CNMA      Accusé de réception nouveau message
AT+CSMP      Paramètres SMS (mode texte)
AT+CNMI      Nouvelles notifications de message
```

— TTY.PY – ABSTRACTION SÉRIE —

```
# Fournit une interface de bas niveau pour le port série
# Utilisé en interne par le terminal gsmterm.py
# Encapsule pySerial pour l'interactivité du terminal
```

— UTILISATION EN TERMINAL INTERACTIF —

```
$ python tools/gsmterm.py -p /dev/ttyUSB0
# Interface readline avec :
# <Tab>      → complétion des commandes AT
# <Flèche>    → historique
# ?AT+CSQ    → aide sur AT+CSQ
# AT+CSQ     → exécution directe
# quit/exit  → quitter
```

— IMPORT PROGRAMMATIQUE —

```
# gsmtermlib est principalement un outil CLI
# Pour l'utilisation programmatique, préférer GsmModem.write()
from tools.gsmtermlib import atcommands
# atcommands.AT_COMMANDS – dict des commandes AT avec métadonnées
```

[util] Patterns avancés et production

```
DESCRIPTION : Patterns robustes pour les applications de production
POURQUOI    : Les modems GSM peuvent être instables – robustesse indispensable.
CONTEXTE     : Gateway SMS, surveillance IoT, alertes industrielles.
```

— CONNEXION ROBUSTE AVEC RETRY —

```
import time, logging
```

```

from gsmmodem.modem import GsmModem
from gsmmodem.exceptions import TimeoutException

def connecter_modem(port="/dev/ttyUSB0", pin=None, tentatives=5):
    for i in range(tentatives):
        try:
            modem = GsmModem(port)
            modem.connect(pin=pin)
            modem.waitForNetworkCoverage(30)
            return modem
        except TimeoutException as e:
            print(f"Tentative {i+1}/{tentatives} : {e}")
            time.sleep(5)
    raise RuntimeError("Impossible de connecter le modem")

```

— GATEWAY SMS COMPLET

```

import time, threading
from gsmmodem.modem import GsmModem, SentSms
from gsmmodem.exceptions import CommandError, TimeoutException, CmsError

def handleSmsRecu(sms):
    print(f"[IN ] {sms.number}: {sms.text}")
    # Réponse automatique
    try:
        modem.sendSms(sms.number, f"Reçu : '{sms.text}'")
    except Exception as e:
        print(f"Erreur envoi réponse : {e}")

def handleRapport(rapport):
    print(f"[RPT] ref={rapport.reference} → {'OK' if rapport.deliveryStatus == 0 else 'FAIL'}")

def handleAppel(appel):
    print(f"[TEL] Appel de {appel.number}, sonnerie #{appel.ringCount}")
    appel.hangup()
    # Rejeter automatiquement

modem = GsmModem(
    "/dev/ttyUSB0",
    smsReceivedCallbackFunc=handleSmsRecu,
    smsStatusReportCallback=handleRapport,
    incomingCallCallbackFunc=handleAppel,
)
modem.connect()
modem.waitForNetworkCoverage()
print(f"Gateway actif – Opérateur: {modem.networkName}, Signal: {modem.signalStrength}")

try:
    while True:
        time.sleep(5)
        print(f"Signal: {modem.signalStrength}/31")
except KeyboardInterrupt:
    modem.close()

```

— ENVOI AVEC GESTION D'ERREURS COMPLÈTE

```

def envoyer_sms_robuste(modem, numero, texte, tentatives=3):
    for i in range(tentatives):
        try:
            sms = modem.sendSms(numero, texte)
            print(f"SMS envoyé (ref={sms.reference})")
            return sms
        except CmsError as e:
            if e.errorCode == 322:
                # Mémoire pleine : nettoyer et réessayer
                modem.write("AT+CMGD=1,4")
            elif i < tentatives - 1:
                time.sleep(2)
            else:
                raise
        except TimeoutException:
            if i < tentatives - 1:
                time.sleep(5)
            else:
                raise
    return None

```

RÉCAPITULATIF DES IMPORTS

— INSTALLATION —

pip install python-gsmmodem-new Fork maintenu, Python 3 ☐

— IMPORTS —

from gsmmodem.modem import GsmModem	Classe principale
from gsmmodem.modem import SentSms	SMS envoyé
from gsmmodem.modem import ReceivedSms	SMS reçu
from gsmmodem.modem import StatusReport	Rapport de livraison
from gsmmodem.modem import IncomingCall	Appel entrant
from gsmmodem.modem import Ussd	Objet réponse USSD
from gsmmodem.exceptions import (	
CommandError,	Erreur commande AT générique
CmeError,	Erreur CME (matériel/SIM)
CmsError,	Erreur CMS (SMS)
TimeoutException,	Réponse non reçue
InvalidStateException,	État invalide
InterruptedException,	Opération interrompue
PinRequiredError,	PIN SIM requis
IncorrectPinError,	PIN SIM incorrect
SmscNumberUnknownError,	SMSC inconnu
EncodingError,	Erreur encodage PDU
)	

— GUIDE DE DÉCISION RAPIDE —

Envoyer un SMS	→ modem.sendSms(dest, texte)
Recevoir des SMS	→ GsmModem(port, smsReceivedCallbackFunc=cb)
Accusé de réception	→ sendSms(..., waitForDeliveryReport=True)
Appel entrant	→ GsmModem(port, incomingCallCallbackFunc=cb)
Appel sortant	→ modem.dial(numero)
Session USSD	→ modem.sendUssd("*100#")
Commande AT brute	→ modem.write("AT+CSQ")
Terminal interactif	→ python tools/gsmterm.py -p /dev/ttyUSBx
Identifier le modem	→ python -m gsmmodem.tools.identify_modem
Envoyer SMS en ligne commande	→ python tools/sendsms.py -p /dev/ttyUSBx -d +33... -m "..."